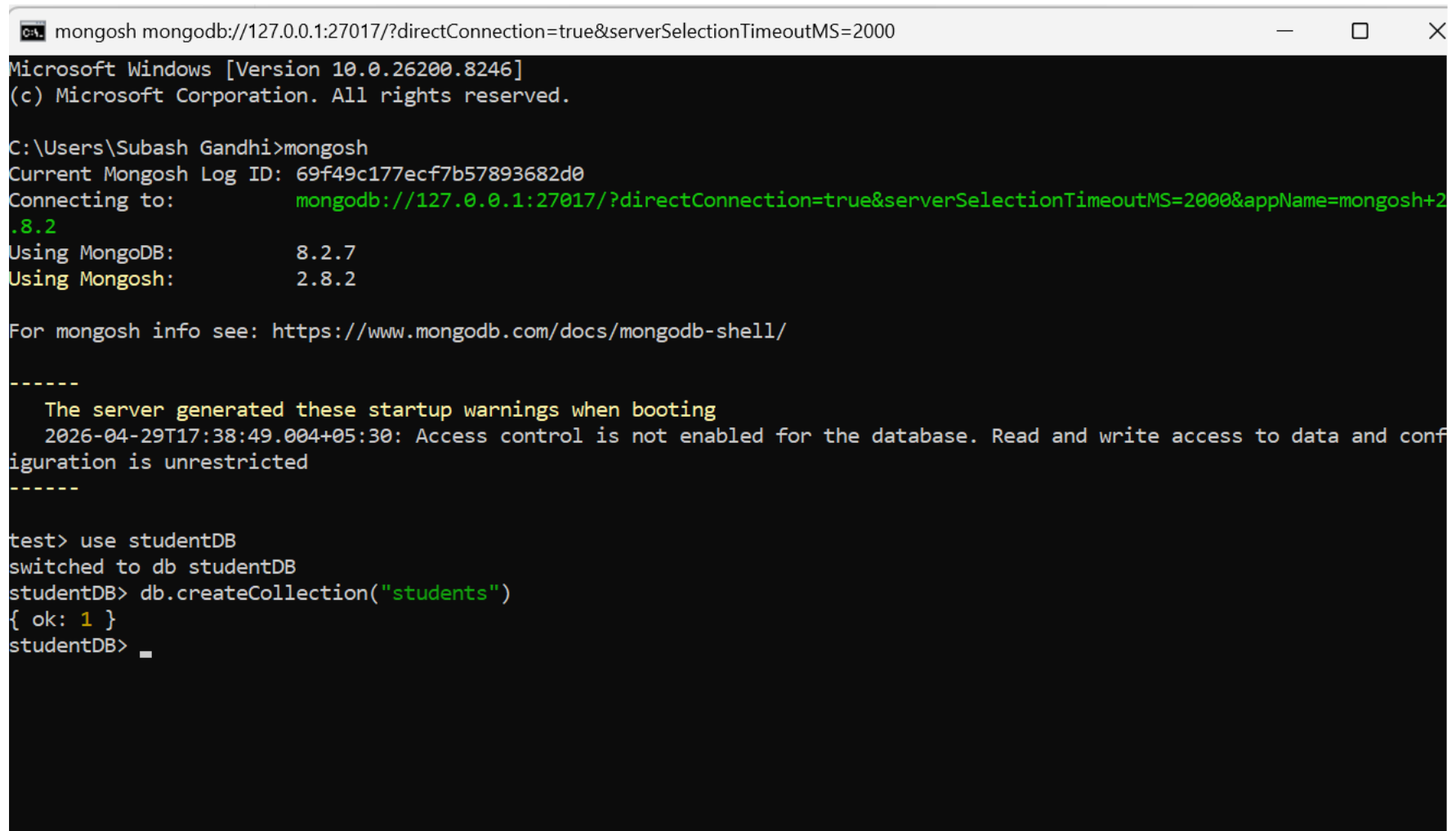


MongoDB Basics and CRUD Operations

1. Create Database and Collection

use studentDB

db.createCollection("students")



```
Microsoft Windows [Version 10.0.26200.8246]
(c) Microsoft Corporation. All rights reserved.

C:\Users\Subash Gandhi>mongosh
Current Mongosh Log ID: 69f49c177ecf7b57893682d0
Connecting to:      mongodb://127.0.0.1:27017/?directConnection=true&serverSelectionTimeoutMS=2000&appName=mongosh+2.8.2
Using MongoDB:      8.2.7
Using Mongosh:      2.8.2

For mongosh info see: https://www.mongodb.com/docs/mongodb-shell/

-----
  The server generated these startup warnings when booting
  2026-04-29T17:38:49.004+05:30: Access control is not enabled for the database. Read and write access to data and configuration is unrestricted
-----

test> use studentDB
switched to db studentDB
studentDB> db.createCollection("students")
{ ok: 1 }
studentDB> _
```

2. Insert Operations

Insert one document:

```
db.students.insertOne({
  name: "Meena",
  age: 25,
  course: "MERN Stack",
  status: "ongoing"
})
```

Insert multiple documents:

```
db.students.insertMany([
  {
    name: "Divya",
    age: 23,
    course: "Python",
    status: "ongoing"
  },
  {
    name: "Swathi",
    age: 21,
    course: "Full stack Developer",
    status: "ongoing"
  },
  {
```

```
name: "kaviya",  
age: 26,  
course: "Java",  
status: "ongoing"  
}  
])
```

```
test> use studentDB  
switched to db studentDB  
studentDB> db.createCollection("students")  
{ ok: 1 }  
studentDB> db.students.insertOne({name:"Meena",age:25,course:"MERN STACK",status:"ongoing"})  
{  
  acknowledged: true,  
  insertedId: ObjectId('69f4a0af0e5b8e9cbc3682d1')  
}  
studentDB> db.students.insertMany([{"name":"Meena",age:25,course:"MERN STACK",status:"ongoing"}, {"name":"Divya",age:23,course:"Python",status:"ongoing"}, {"name":"Swathi",age:21,course:"Full Stack Developer",status:"ongoing"}, {"name":"Kaviya",age:26,course:"Java",status:"ongoing"}])  
{  
  acknowledged: true,  
  insertedIds: {  
    '0': ObjectId('69f4a1d10e5b8e9cbc3682d2'),  
    '1': ObjectId('69f4a1d10e5b8e9cbc3682d3'),  
    '2': ObjectId('69f4a1d10e5b8e9cbc3682d4'),  
    '3': ObjectId('69f4a1d10e5b8e9cbc3682d5')  
  }  
}  
studentDB> _
```

3. Read Operations

Fetch all documents:

```
db.students.find()
```

```
studentDB> db.students.find()  
[  
  {  
    _id: ObjectId('69f4a1d10e5b8e9cbc3682d2'),  
    name: 'Meena',  
    age: 25,  
    course: 'MERN STACK',  
    status: 'ongoing'  
  },  
  {  
    _id: ObjectId('69f4a1d10e5b8e9cbc3682d3'),  
    name: 'Divya',  
    age: 23,  
    course: 'Python',  
    status: 'ongoing'  
  },  
  {  
    _id: ObjectId('69f4a1d10e5b8e9cbc3682d4'),  
    name: 'Swathi',  
    age: 21,  
    course: 'Full Stack Developer',  
    status: 'ongoing'  
  },  
  {  
    _id: ObjectId('69f4a1d10e5b8e9cbc3682d5'),  
    name: 'Kaviya',  
    age: 26,  
    course: 'Java',  
    status: 'ongoing'  
  }  
]
```

Fetch students enrolled in MERN Stack:

```
db.students.find({ course: "MERN Stack" })
```

```

]
studentDB> db.students.find({course:"MERN STACK"})
[
  {
    _id: ObjectId('69f4a1d10e5b8e9cbc3682d2'),
    name: 'Meena',
    age: 25,
    course: 'MERN STACK',
    status: 'ongoing'
  }
]
studentDB>

```

4. Update Operations

Update one document:

```

db.students.updateOne(
  { name: "Meena" },
  { $set: { status: "completed" } }
)

```

```

studentDB> db.students.updateOne({name:"Meena"},{$set:{status:"completed"}})
{
  acknowledged: true,
  insertedId: null,
  matchedCount: 1,
  modifiedCount: 1,
  upsertedCount: 0
}
studentDB> db.students.find()
[
  {
    _id: ObjectId('69f4a1d10e5b8e9cbc3682d2'),
    name: 'Meena',
    age: 25,
    course: 'MERN STACK',
    status: 'completed'
  },
]

```

Update multiple documents:

I added one more students {name:"Arun",age:20,course:"Full Stack Developer",status:"ongoing"}

```

db.students.updateMany(
  { course: "Full Stack Developer" },
  { $set: { status: "completed" } }
)

```

```

},
{
  _id: ObjectId('69f4a1d10e5b8e9cbc3682d3'),
  name: 'Divya',
  age: 23,
  course: 'Python',
  status: 'ongoing'
},
{
  _id: ObjectId('69f4a1d10e5b8e9cbc3682d4'),
  name: 'Swathi',
  age: 21,
  course: 'Full Stack Developer',
  status: 'completed'
},
{
  _id: ObjectId('69f4a1d10e5b8e9cbc3682d5'),
  name: 'Kaviya',
  age: 26,
  course: 'Java',
  status: 'ongoing'
},
{
  _id: ObjectId('69f4ac6c0e5b8e9cbc3682d7'),
  name: 'Arun',
  age: 20,
  course: 'Full Stack Developer',
  status: 'completed'
}
]

```

5. Delete Operations

Delete one document:

```
db.students.deleteOne({ name: "Divya" })
```

```
studentDB> db.students.deleteOne({name:"Divya"})
{ acknowledged: true, deletedCount: 1 }
studentDB> db.students.find()
[
  {
    _id: ObjectId('69f4a1d10e5b8e9cbc3682d2'),
    name: 'Meena',
    age: 25,
    course: 'MERN STACK',
    status: 'completed'
  },
  {
    _id: ObjectId('69f4a1d10e5b8e9cbc3682d4'),
    name: 'Swathi',
    age: 21,
    course: 'Full Stack Developer',
    status: 'completed'
  },
  {
    _id: ObjectId('69f4a1d10e5b8e9cbc3682d5'),
    name: 'Kaviya',
    age: 26,
    course: 'Java',
    status: 'ongoing'
  },
  {
    _id: ObjectId('69f4ac6c0e5b8e9cbc3682d7'),
    name: 'Arun',
    age: 20,
    course: 'Full Stack Developer',

```

Delete all documents:

```
db.students.deleteMany({})
```

```
studentDB> db.students.deleteMany({course:"Full Stack Developer"})
{ acknowledged: true, deletedCount: 2 }
studentDB> db.students.find()
ReferenceError: students is not defined
studentDB> db.students.find()
[
  {
    _id: ObjectId('69f4a1d10e5b8e9cbc3682d2'),
    name: 'Meena',
    age: 25,
    course: 'MERN STACK',
    status: 'completed'
  },
  {
    _id: ObjectId('69f4a1d10e5b8e9cbc3682d5'),
    name: 'Kaviya',
    age: 26,
    course: 'Java',
    status: 'ongoing'
  }
]
studentDB>
```

6. Query Operators

These are done before delete operation.

Greater than:

```
db.students.find({ age: { $gt: 23 } })
```

```
studentDB> db.students.find({age:{$gt:23}})
[
  {
    _id: ObjectId('69f4a1d10e5b8e9cbc3682d2'),
    name: 'Meena',
    age: 25,
    course: 'MERN STACK',
    status: 'completed'
  },
  {
    _id: ObjectId('69f4a1d10e5b8e9cbc3682d5'),
    name: 'Kaviya',
    age: 26,
    course: 'Java',
    status: 'ongoing'
  }
]
studentDB>
```

Less than:

```
db.students.find({ age: { $lt: 25 } })
```

```
]
studentDB> db.students.find({age:{$lt:25}})
[
  {
    _id: ObjectId('69f4a1d10e5b8e9cbc3682d3'),
    name: 'Divya',
    age: 23,
    course: 'Python',
    status: 'ongoing'
  },
  {
    _id: ObjectId('69f4a1d10e5b8e9cbc3682d4'),
    name: 'Swathi',
    age: 21,
    course: 'Full Stack Developer',
    status: 'completed'
  },
  {
    _id: ObjectId('69f4ac6c0e5b8e9cbc3682d7'),
    name: 'Arun',
    age: 20,
    course: 'Full Stack Developer',
    status: 'completed'
  }
]
studentDB> _
```

Using \$in:

```
db.students.find({ course: { $in: ["MERN Stack", "Python"] } })
```

```
]
studentDB> db.students.find({course:{$in:["MERN STACK","Python"]}})
[
  {
    _id: ObjectId('69f4a1d10e5b8e9cbc3682d2'),
    name: 'Meena',
    age: 25,
    course: 'MERN STACK',
    status: 'completed'
  },
  {
    _id: ObjectId('69f4a1d10e5b8e9cbc3682d3'),
    name: 'Divya',
    age: 23,
    course: 'Python',
    status: 'ongoing'
  }
]
studentDB> _
```

Using \$and:

```
db.students.find({
```

```
$and: [
```

```
{ course: "MERN Stack" },
```

```
{ status: "completed" }
```

```
]
```

```
})
```

```
studentDB> db.students.find({$and:[{course:"MERN STACK"},{status:"completed"}]})
[
  {
    _id: ObjectId('69f4a1d10e5b8e9cbc3682d2'),
    name: 'Meena',
    age: 25,
    course: 'MERN STACK',
    status: 'completed'
  }
]
studentDB> _
```

Using \$or:

```
db.students.find({
```

```
$or: [
```

```
{ course: "MERN Stack" },
```

```
{ age: { $lt: 23 }}
```

```
]
```

```
}}
```

```
]
studentDB> db.students.find({$or:[{course:"MERN STACK"},{age:{$lt:23}}]})
[
  {
    _id: ObjectId('69f4a1d10e5b8e9cbc3682d2'),
    name: 'Meena',
    age: 25,
    course: 'MERN STACK',
    status: 'completed'
  },
  {
    _id: ObjectId('69f4a1d10e5b8e9cbc3682d4'),
    name: 'Swathi',
    age: 21,
    course: 'Full Stack Developer',
    status: 'completed'
  },
  {
    _id: ObjectId('69f4ac6c0e5b8e9cbc3682d7'),
    name: 'Arun',
    age: 20,
    course: 'Full Stack Developer',
    status: 'completed'
  }
]
studentDB> 
```

Using \$exists:

This is done after deletion

```
db.students.find({ status: { $exists: true } })
```

```
studentDB> db.students.find({status:{$exists:true}})
[
  {
    _id: ObjectId('69f4a1d10e5b8e9cbc3682d2'),
    name: 'Meena',
    age: 25,
    course: 'MERN STACK',
    status: 'completed'
  },
  {
    _id: ObjectId('69f4a1d10e5b8e9cbc3682d5'),
    name: 'Kaviya',
    age: 26,
    course: 'Java',
    status: 'ongoing'
  }
]
studentDB> 
```

Conclusion

MongoDB is a NoSQL database that stores data in document format.

A database contains collections, and collections contain documents.

Using MongoDB CRUD operations, we can insert, read, update, and delete data easily.

Query operators like \$gt, \$lt, \$in, \$and, \$or, and \$exists help us filter data more effectively.